



RADU POPESCU

**IMPROVING THE WRITE SCALABILITY
OF THE CERNVM FILE SYSTEM
WITH ERLANG/OTP**

THE LARGE HADRON COLLIDER

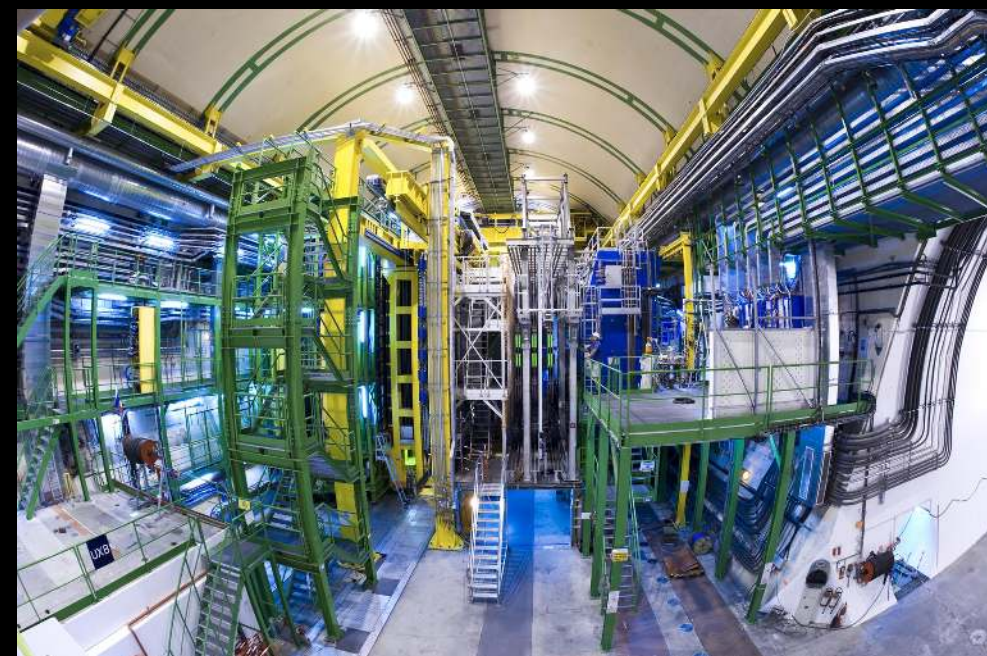
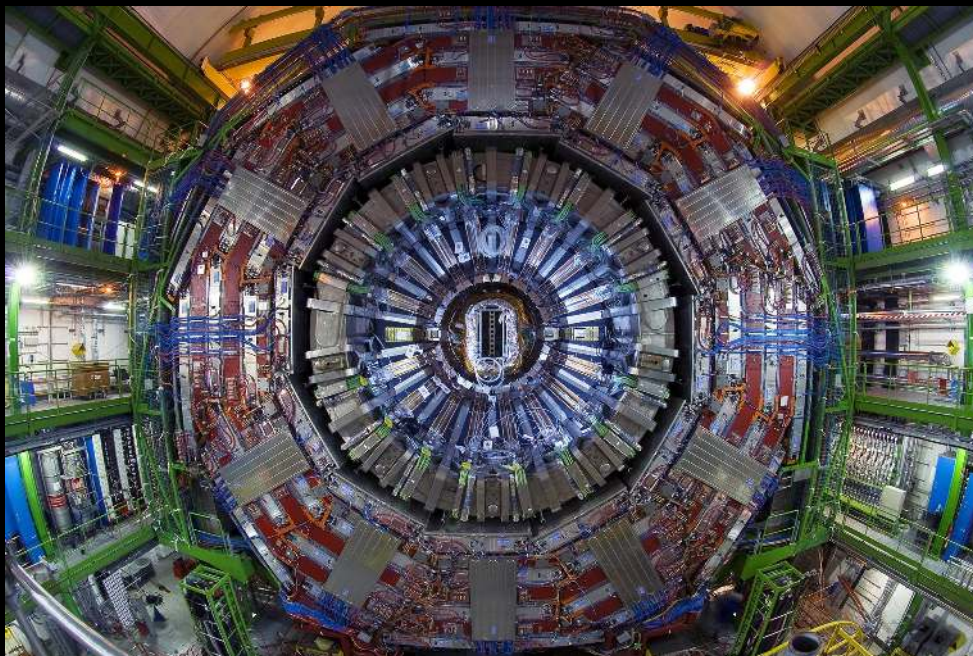
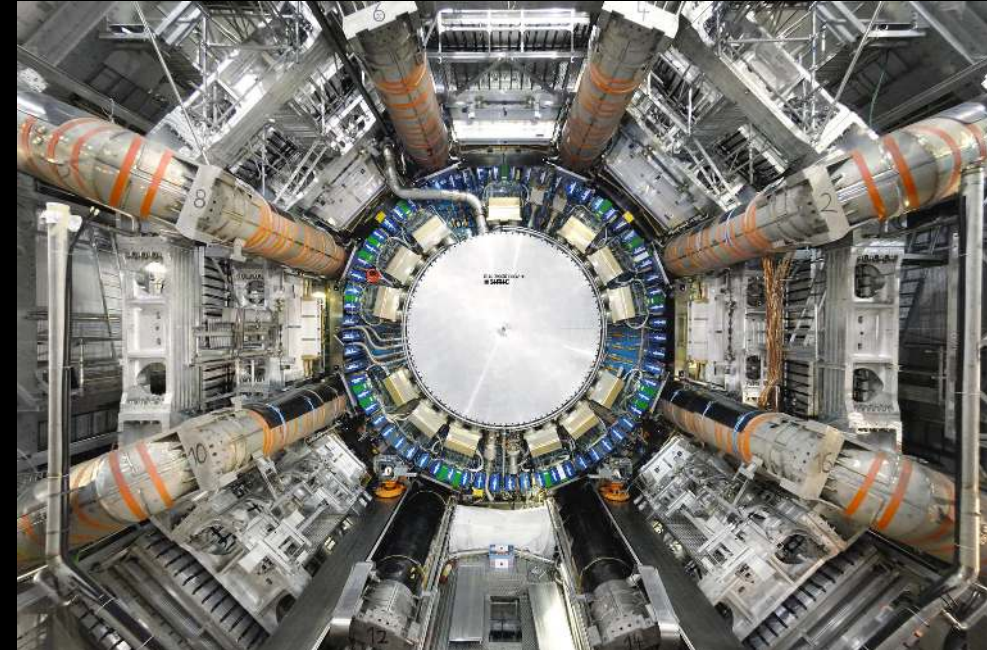
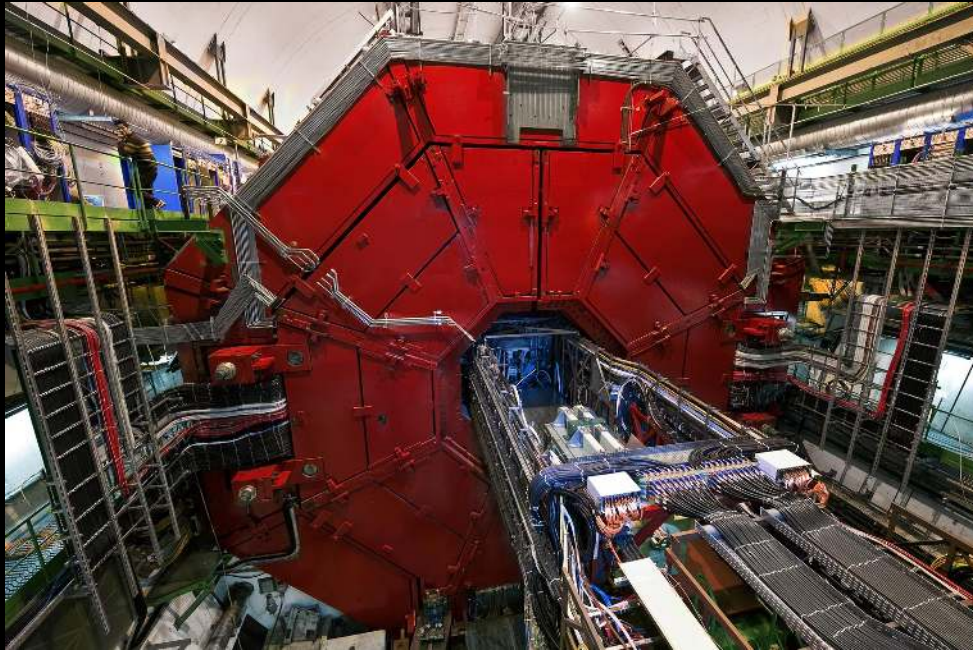


TUNNEL VISION

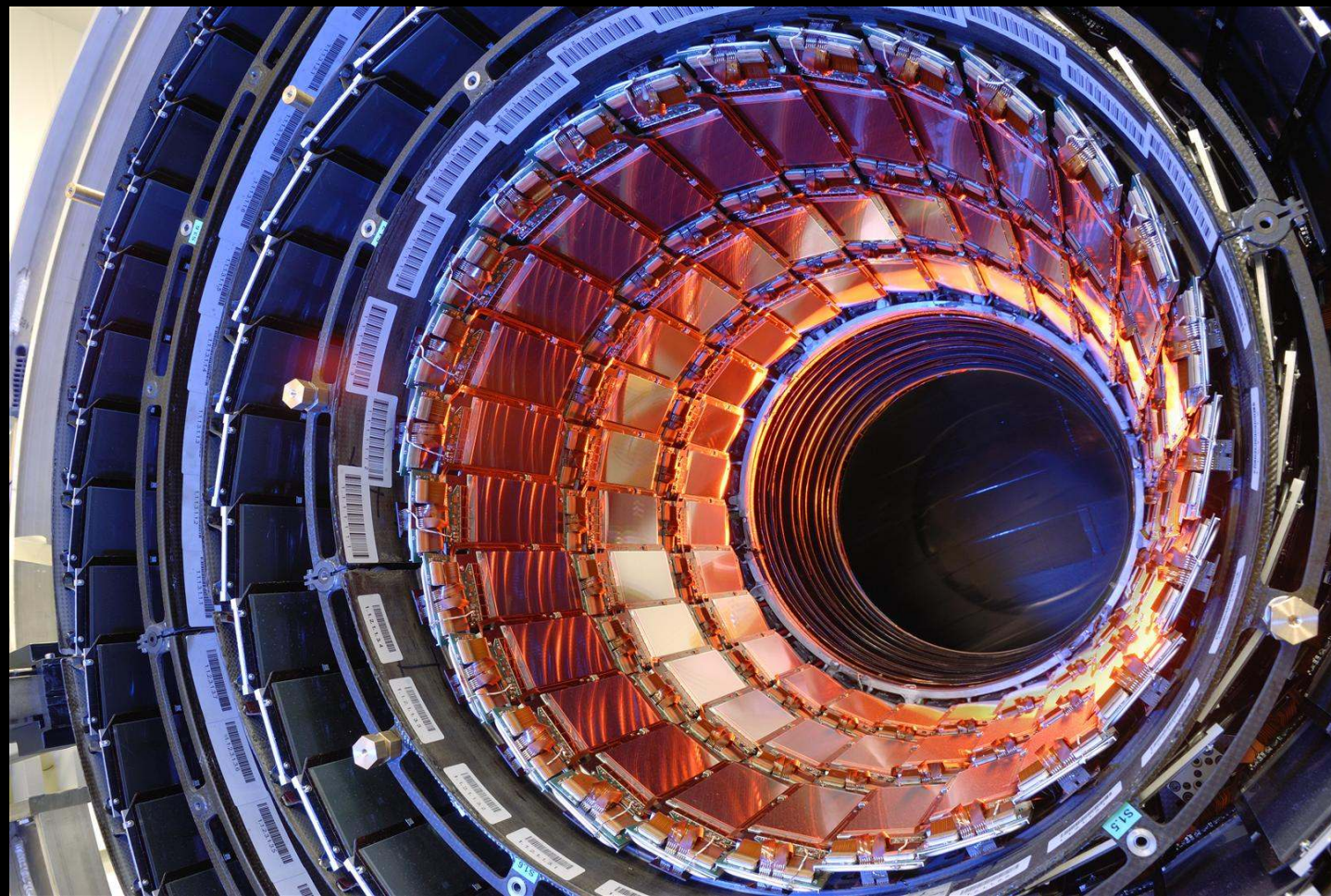
- ▶ 27 km circumference
- ▶ 100 m underground
- ▶ 180MW power consumption
- ▶ 7 TeV per beam



ALICE, ATLAS, CMS AND LHCb DETECTORS



CMS DETECTOR INNER BARREL



SUPER COLLIDER

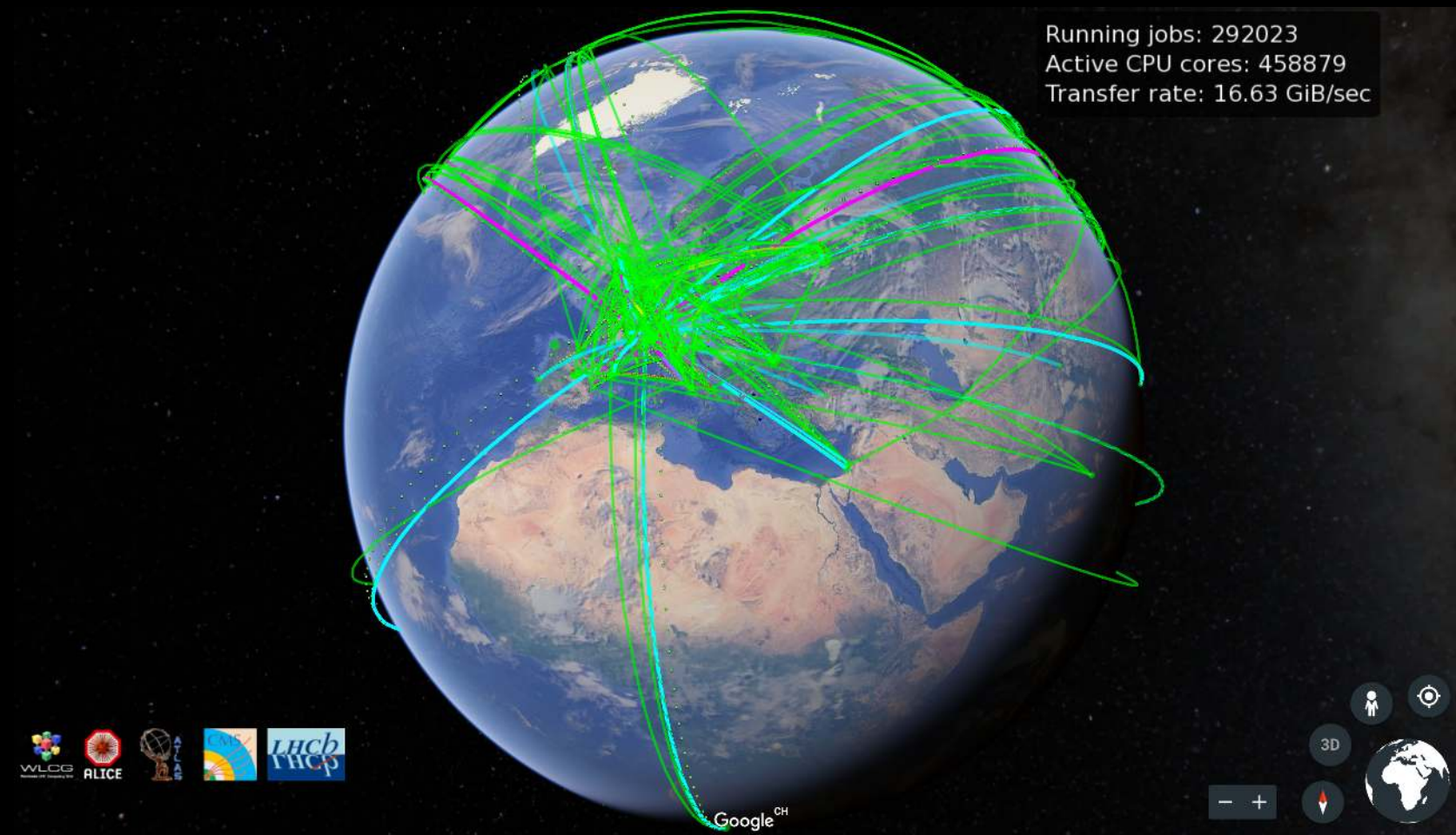


Super Collider, Mustaine et al. (2013, Universal Records)

EXPERIMENT DATA CHALLENGE

- ▶ 100 Million channels, bunch crossing every 25 ns
- ▶ 1 PB/s internal data rate
- ▶ 5 PB data / year recorded (plus derived data sets)
- ▶ 100 PB / year by 2025 (x20)
- ▶ 5 million lines of code / experiment

GLOBALLY DISTRIBUTED



[Worldwide LHC Compute Grid live map](#)

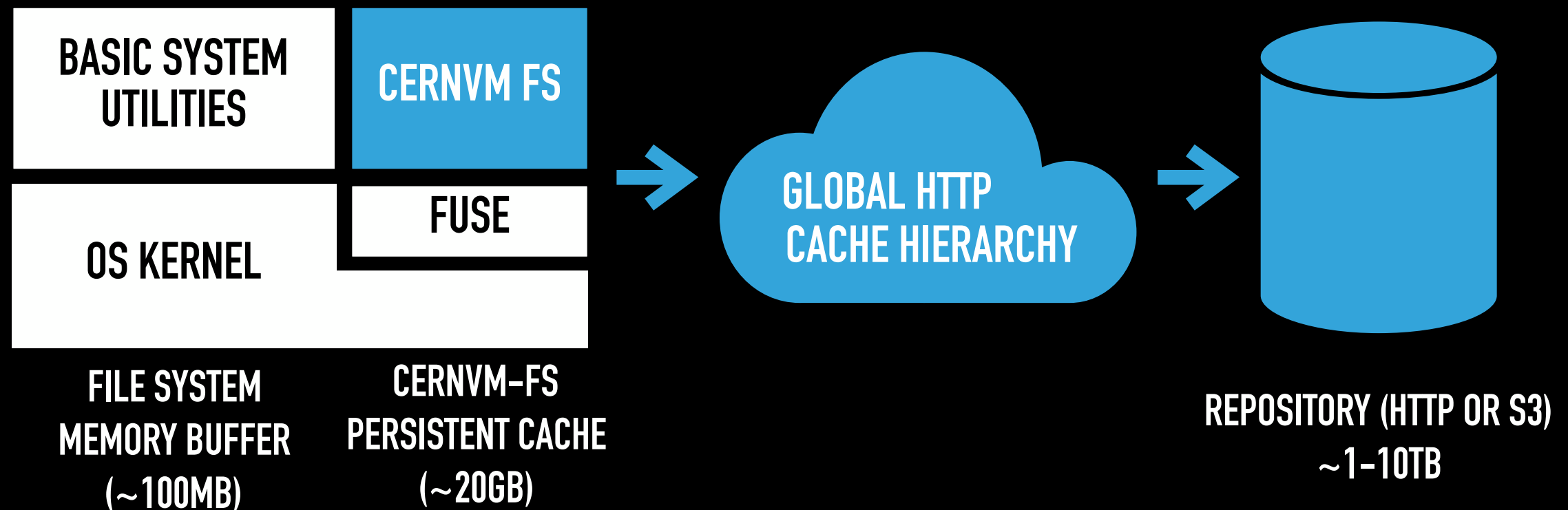
42 countries, 170 computing centres, 2 million jobs run each day

KEY FIGURES

- ▶ Hundreds of developers
- ▶ $\sim 10^8$ binaries
- ▶ $\sim 1\text{TB}$ / day of nightly builds
- ▶ $\sim 100\,000$ machines world-wide
- ▶ Daily production releases, remain available

THE CERNVM FILE SYSTEM

A FILE SYSTEM APPROACH TO DISTRIBUTING SOFTWARE



- ▶ ~100 000 clients
- ▶ FUSE based, independent mount points, e. g. /cvmfs/atlas.cern.ch
- ▶ Clients have a read-only view; single writer into repository
- ▶ HTTP transport, access and caching on demand

MAIN COMPONENTS

- ▶ Client: FUSE module (with cache plugins)
- ▶ Server tools (command line tools)
- ▶ Standard HTTP server
- ▶ HTTP caches

DESIGN

- ▶ Data store:
 - ▶ Immutable content-addressed blobs (*)
 - ▶ Compression, deduplication
- ▶ Metadata:
 - ▶ Catalogs: state of the entire repository at a given moment in time is encoded in a Merkle tree
 - ▶ Digitally signed manifest
- ▶ Versioning, snapshots etc.
- ▶ PULL based!

CVMFS PUBLICATION WORKFLOW

PUBLISHING

- ▶ Single writer (stateless command line utilities)
- ▶ A read/write view is constructed with a union mount (OverlayFS, Aufs)
- ▶ Files are compressed and hashed, and written to repository storage
- ▶ New metadata catalogs are created and published
- ▶ Repository manifest is updated (atomic operation)

EXISTING WORKFLOW

- ▶ Centralised release manager machine
- ▶ Direct interaction with the release manager:

```
$ ssh my-cvmfs-server.cern.ch
```

```
$ cvmfs_server transaction
```

```
$ vim /cvmfs/my-cvmfs-server.cern.ch/some_file.org
```

... (Make changes to files in the R/W mount)

```
$ cvmfs_server publish
```


EXISTING WORKFLOW

PROS:

- ▶ Straightforward to use
- ▶ Good for scripting
- ▶ Hides somewhat the distributed nature of the system

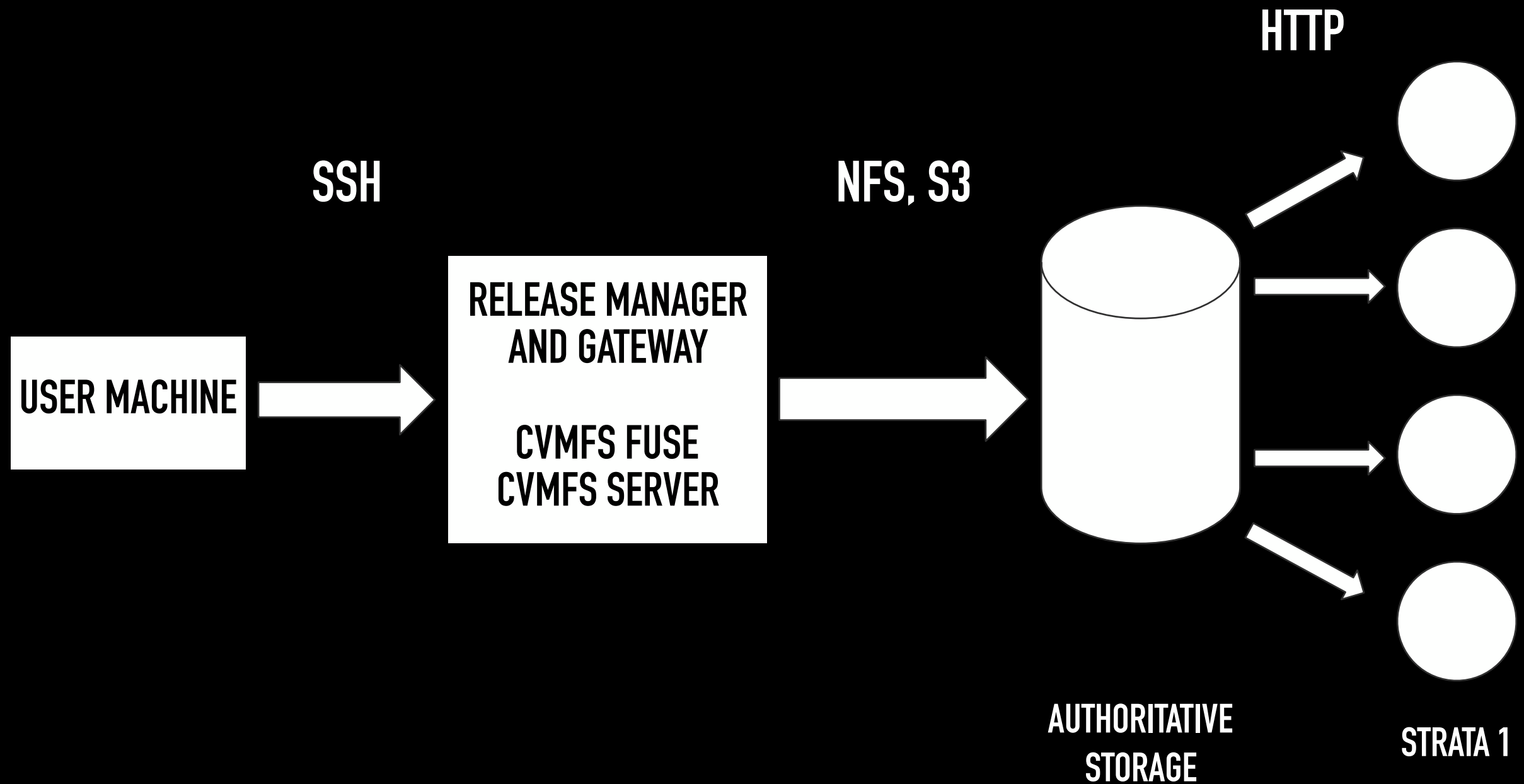
CONS:

- ▶ No support for concurrent writing
- ▶ Can be unsafe (shell access to machine with repository storage)
- ▶ Performance issues for large change-sets

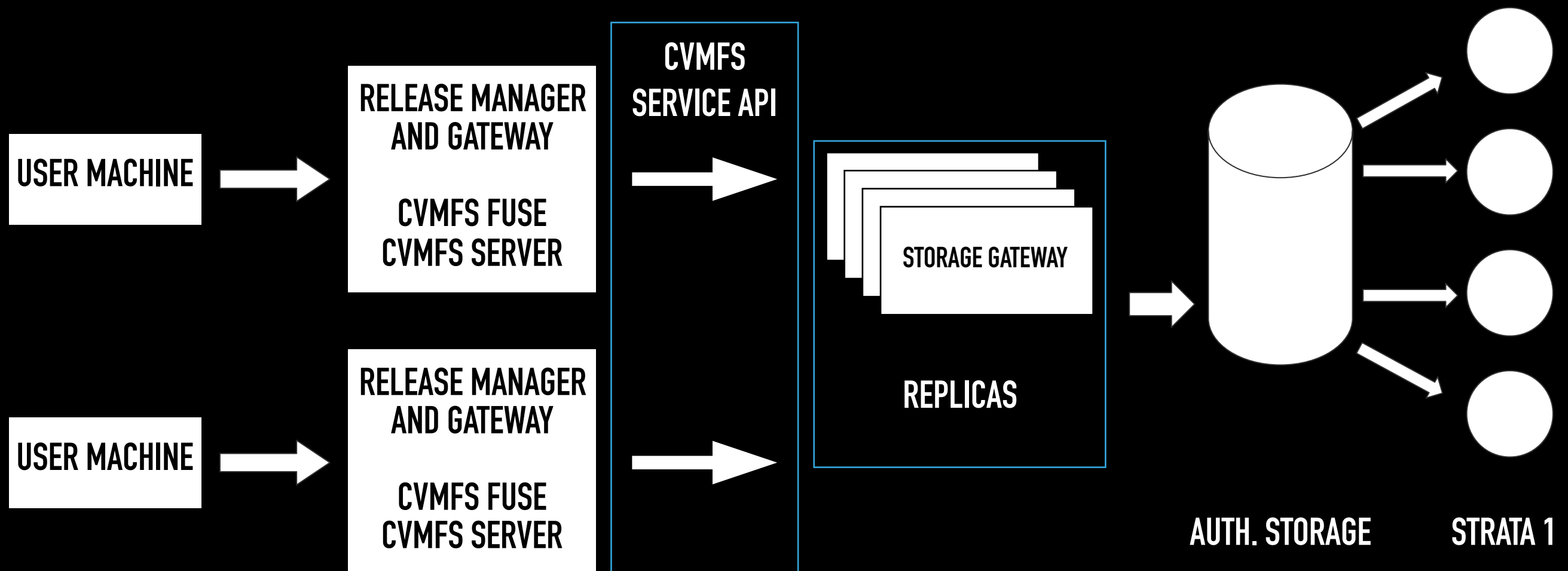
PROPERTIES AND CONSTRAINTS

1. The system (repository + cache + clients) is eventually consistent
2. Concurrency can be further exploited due to:
 - ▶ Immutability of CAS
 - ▶ Pushing objects is idempotent
 - ▶ Directory tree structure
3. The critical section involves updating the metadata catalog and swapping the manifest

EXISTING ARCHITECTURE



AN IMPROVED ARCHITECTURE



AN IMPROVED WORKFLOW

```
$ ssh my-cvmfs-1.cern.ch
```

```
$ cvmfs_server transaction /lcg/58
```

... (Make changes to files in the R/W mount)

```
$ vim /cvmfs/my-cvmfs.cern.ch/lcg/58/  
some_file.org
```

```
$ cvmfs_server publish
```

```
$ ssh my-cvmfs-2.cern.ch
```

```
$ cvmfs_server transaction /lcg/60
```

... (Make changes to files in the R/W mount)

```
$ vim /cvmfs/my-cvmfs.cern.ch/lcg/60/  
some_file.org
```

```
$ cvmfs_server publish
```

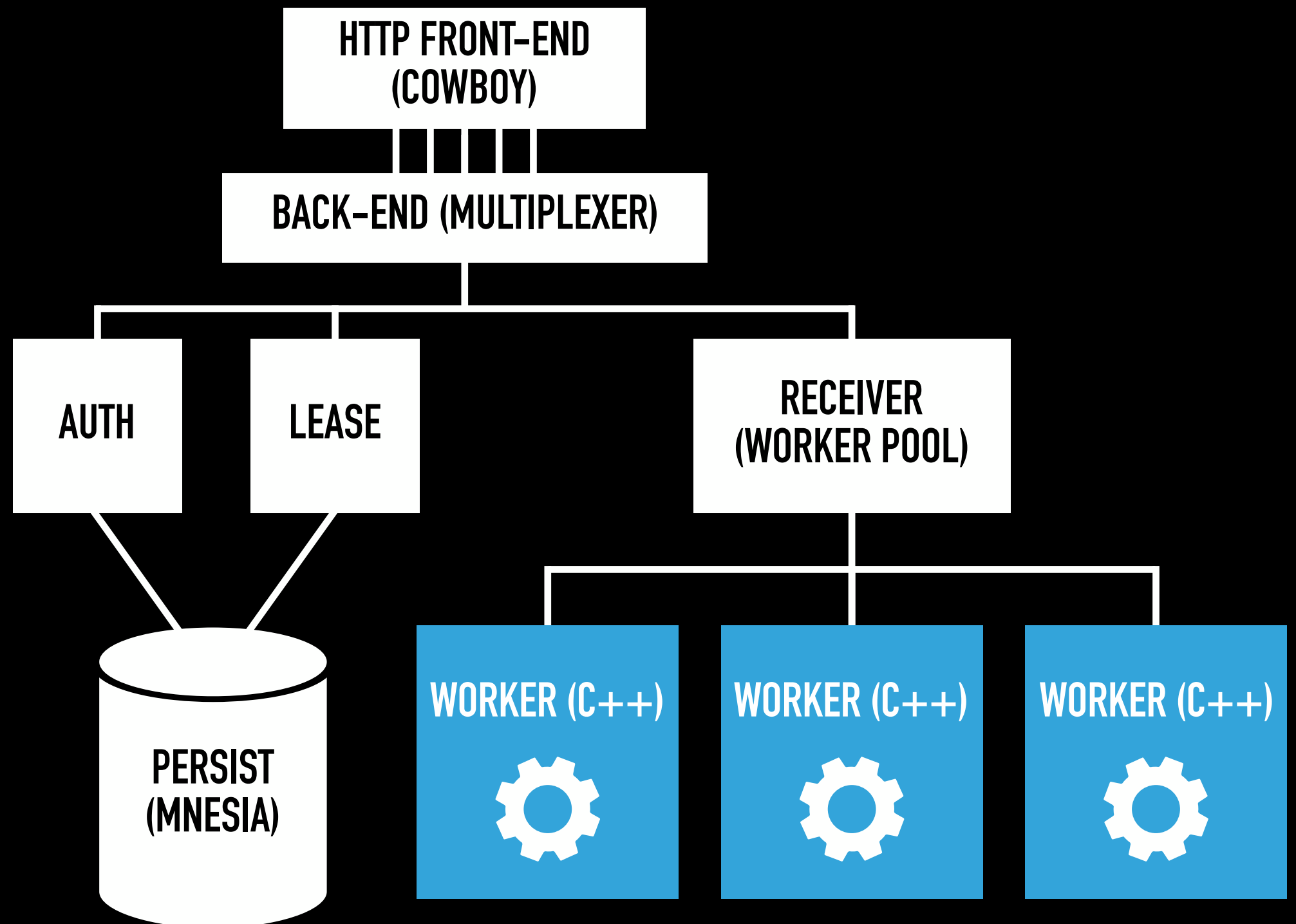
CVMFS STORAGE GATEWAY

- ▶ Serves as a distributed lock manager
 - ▶ Checks rights of clients to modify repositories
 - ▶ Assigns exclusive leases to clients on repository subpaths
- ▶ Receives files (object packs) from clients, writes them to authoritative storage

ERLANG/OTP: DISTRIBUTED GLUE

- ▶ Language (Erlang) and framework (OTP) designed for concurrent and distributed applications:
 - ▶ Actor model: lightweight processes with memory isolation
 - ▶ Immutability of values
 - ▶ Supervision trees
 - ▶ Erlang/OTP/BEAM are battle-tested, 30+ years of use at Ericsson
- ▶ Excellent C/C++ interoperability

GATEWAY APPLICATION ARCHITECTURE



DEVELOPER EXPERIENCE WITH ERLANG/OTP

Great:

- ▶ OTP
- ▶ Tracing, inspection etc.
- ▶ Immutability, Functional language
- ▶ Very simple to write concurrent programs
- ▶ Use Dialyzer, CommonTest, QuickCheck etc.
- ▶ Easy integration with C++

DEVELOPER EXPERIENCE WITH ERLANG/OTP

Less great:

- ▶ Dynamic typing is strange, coming from C++
- ▶ Deciphering Erlang errors is an acquired taste (use Lager for logging)
- ▶ Large APIs in OTP, some parts feel less clearly documented

DEVELOPER EXPERIENCE WITH ERLANG/OTP

- ▶ Overall impression is very positive!
- ▶ Would definitely use it for other new components
- ▶ Looking forward to more operational experience

DOCKER GRAPHDRIVER PLUGIN

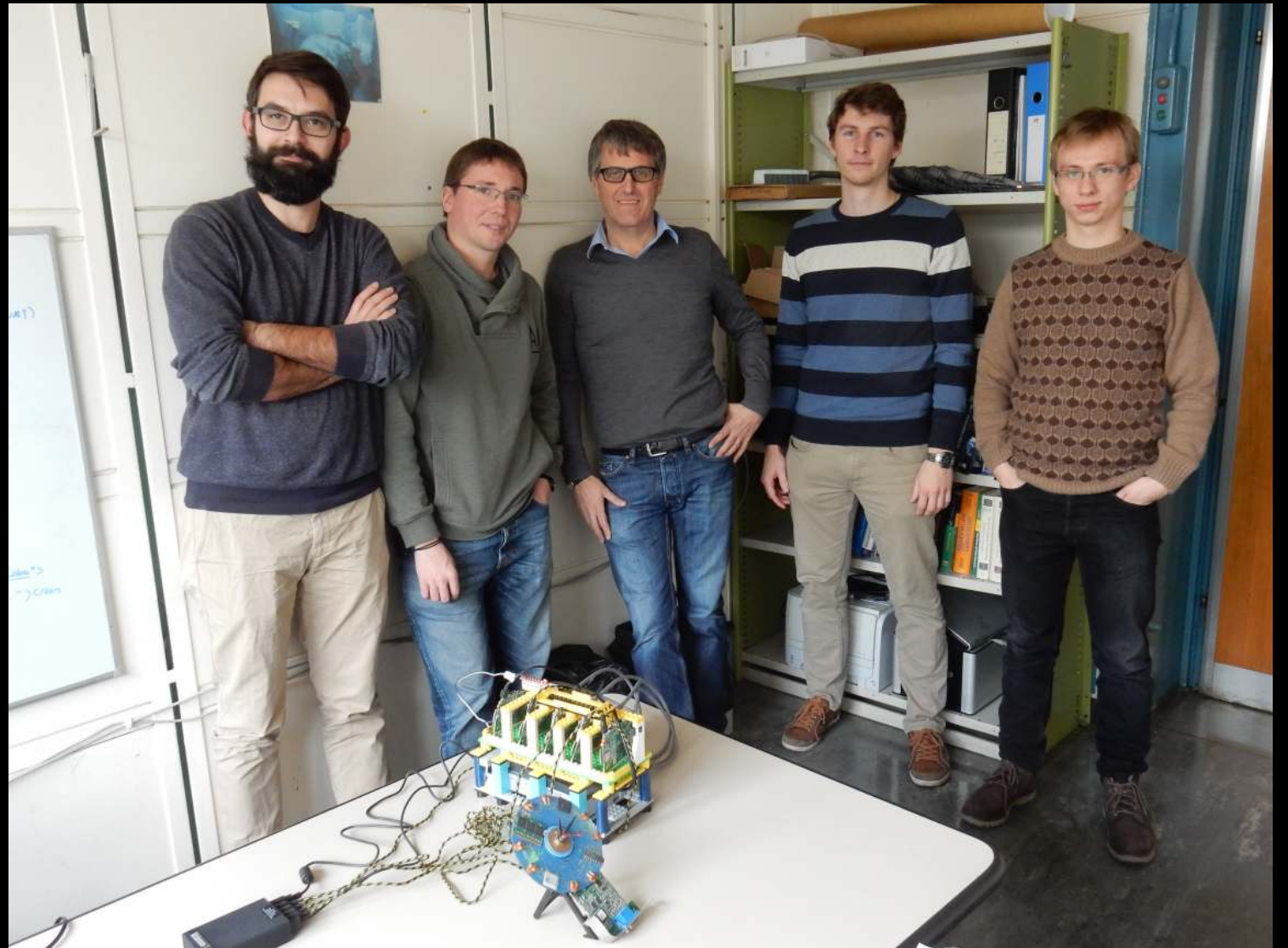
- ▶ Docker Graphdriver plugin for CernVM-FS (Nikola Hardi):
<https://github.com/cvmfs/docker-graphdriver>
- ▶ Store the contents of Docker image layers inside CernVM FS repositories
- ▶ Instead of having to download the entire layers, mount a CernVM FS repository and download individual files on-demand

CERN VM 10TH ANNIVERSARY!

- ▶ Next year, CernVM is turning 10
- ▶ Jan 30th -> Feb 1st 2018: CernVM workshop @CERN
- ▶ Open to anyone
- ▶ Talks by users and developers of CernVM and related projects

THE CERNVM TEAM (LEFT TO RIGHT)

- ▶ Radu Popescu
- ▶ Jakob Blomer
- ▶ Gerardo Ganis
- ▶ Petr Jirout (former)
- ▶ Nikola Hardi (former)



THANK YOU

CernVM-FS:

<https://github.com/cvmfs>

<http://cvmfs.readthedocs.io/en/stable/>

radu.popescu@cern.ch,
<https://github.com/radupopescu>,
@iradupopescu

CRITICAL SECTIONS

- ▶ Erlang (only) provides processes and message passing for concurrency
- ▶ No locks, semaphores, condition variables etc.
- ▶ What if a exclusive access to a resource is needed?
- ▶ OTP `gen_server` works as a critical section

MULTIPLEXING REQUESTS/REPLIES ON GEN_SERVER

- ▶ OTP `gen_server` with concurrency?
- ▶ In `gen_server:handle_call`, spawn a process per request, and return `{noreply, ...}`
- ▶ The spawned process later returns a value with `gen_server:reply`.
- ▶ Does not maintain order of requests
- ▶ “Concurrency adaptor” between Cowboy and C++ worker pool